

17. REST - cesty, CRUD, HATEOAS

REST

- Representational State Transfer
- Komunikace přes HTTP protokol (mezi různými systémy)
- REST api je api využívající REST architekturu
- Principy:
 - Bezstavovost – požadavek má všechny info k celému zpracování
 - Klient/server architektura
 - Cacheovatelnost – odpověď může být cachována pro znovuvyužití
 - Vrstvený systém - Klient nemusí vědět, zda komunikuje přímo se serverem nebo s mezivrstvou
 - Code-on-Demand – server může zaslat klientovy kód pro rozšíření funkčnosti
- HTTP metody – GET, POST, PUT, DELETE
- Uniformní rozhraní – jednotné a konzistentní pojmenovávání
- Příklad url na get usera s id 1: curl -X GET "https://api.example.com/users/1"
- Výhody: jednoduché, škálovatelné, široce podporované
- Nevýhody: moc/málo dat u složitých operací, verzování

CRUD

- Základní operace pro dlouhodobé (persistentní) ukládání dat
- C – Create, R – Read, U – Update, D – Delete
- Odpovídají základním HTTP metodám (create = post, read = get, update = put)
- Dají se poté pozměňovat a upravovat například podle vstupu (např. readAll = přečte všechny, read = přečte jen daný jeden záznam)

HATEOAS

- Hypermedia as the Engine of Application State
- Architektonický princip rozšiřující REST
- Rozšíření o hypermediální ovládací prvky, které umožňují klientům dynamicky navigovat k dostupným zdrojům
- Nezávislá operace klienta a serveru
- Klient nemusí znát strukturu API, pouze vstupní bod
- Klienti mohou snadněji objevit další data a možnosti
- Lepší verzování a evoluce

```
{
  "id": 1,
  "name": "John",
  "_links": {
    "self": "https://api.example.com/users/1",
    "edit": "https://api.example.com/users/1/edit"
  }
}
```

Klient může následovat odkaz "edit" k editaci uživatele